30538 Problem Set 1: git

"This submission is my work alone and complies with the 30538 integrity policy." Add your initials to indicate your agreement:XH

Before you begin this problem set, please rename `ps1_template.qmd` as `ps1.qmd`.

Name: Somi(Xinyu) Hu UChicago ID (`xxx@uchicago.edu`): xinyuhu

SOLO

Learn git branching (15 points)

Go to https://learngitbranching.js.org. This is the best visual git explainer we know of.

1. Complete all the levels of main "Introduction Sequence". Report the commands needed to complete "Git rebase" with one line per command.

```
git branch bugFix
git checkout bugFix
git commit
git checkout main
git commit
git rebase main bugFix
```

2. Complete all the levels of main "Ramping up". Report the commands needed to complete "Reversing changes in git" with one line per command.

```
git reset local~1
git checkout pushed
git revert pushed
```

3. Complete all the levels of remote "Push & Pull – Git Remotes!". Report the commands needed to complete "Locked Main" with one line per command.

```
git branch feature
git reset --hard o/main
git checkoyt feature
git push
```

Exercises

- Basic Staging and Branching (10-15)

1. Exercise. For your pset submission, tell us only the answer to the last question (22).

```
nothing to commit, working tree clean
```

2. Exercise. For your pset submission, tell us only the output to the last question (18).

```
diff --git a/file1.txt b/file1.txt
deleted file mode 100644
index b376f1e..0000000
--- a/file1.txt
+++ /dev/null
@@ -1 +0,0 @@
-Somi
diff --git a/file2.txt b/file2.txt
new file mode 100644
index 0000000..e69de29
```

- Merging

1. Exercise. After completing all the steps (1 through 12), run git log –oneline –graph –all and report the output.

```
* 1933c36 (HEAD -> master, feature/uppercase) add greeting.txt
* d00f9f0 Add content to greeting.txt
* 65088c4 Add file greeting.txt
```

2. Exercise. Report the answer to step 11.

```
*   8d06655 (HEAD -> master) Merge branch 'greeting'
|\
| * e3efdeb (greeting) add greeting.txt
* | e198a77 add README.md
|/
* f28d9d1 Add content to greeting.txt
* 3a9554e Add file greeting.txt
```

3. Identify the type of merge used in Q1 and Q2 of this exercise. In words, explain the difference between the two merge types, and describe scenarios where each type would be most appropriate. Q1 uses a fast-forward merge. Since the master branch had no new commits after the creation of the feature/uppercase branch, Git simply moves the master branch pointer to the latest commit of that branch. This type of merge does not create an additional merge commit. This method is best suited for individual development because it keeps the commit history extremely clean and linear.

In Q2, because new content (README.md) was also created and committed on the master branch, the histories of the two branches diverged. Therefore, a three-way merge was performed. Git finds the common ancestor of the two branches and merges the changes from both sides, automatically generating a merge commit with two parent nodes. This method is more suitable for scenarios requiring parallel collaboration, such as team collaboration, because it clearly and completely preserves the information.

- Undo, Clean, and Ignore

1. Exercise. Report the answer to step 13.

```
commit 85fb71ca414bf6ff603e64ad0a2e1107528bf4c6
Author: git-katas trainer bot <git-katas@example.com>
Date:   Thu Jan 8 17:27:59 2026 -0600

    Add credentials to repository

diff --git a/credentials.txt b/credentials.txt
new file mode 100644
index 0000000..8995708
--- /dev/null
+++ b/credentials.txt
@@ -0,0 +1 @@
+supersecretpassword
```

2. Exercise. Look up `git clean` since we haven't seen this before. For context, this example is about cleaning up compiled C code, but the same set of issues apply to random files generated by knitting a document or by compiling in Python. Report the terminal output from step 7.

```
Removing README.txt~
Removing obj/
Removing src/myapp.c~
Removing src/oldfile.c~
```

3. Exercise. Report the answer to 15 ("What does git status say?")

```
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    deleted:    file1.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   .gitignore

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    file3.txt
```